🎯 Event Management API - Documentation

📦 Setup & Installation

1. Setup Database

```bash
# Login ke MySQL
mysql -u root -p

# Jalankan SQL schema
mysql -u root -p < setup.sql
```

2. Konfigurasi API

Edit bagian konfigurasi di `api.php`:

```php
private $host = 'localhost';
private $dbName = 'event_management';
private $username = 'root';
private $password = 'your_password';
```

3. Setup Server

```bash
```

```
# Letakkan api.php di folder web server
# Contoh: /var/www/html/ atau htdocs/

# Test API
php -S localhost:8000 api.php
```

🚀 API Endpoints

Base URL

```
http://localhost:8000/api.php
```

1. GET All Events

```bash
# Ambil semua event
curl http://localhost:8000/api.php

# Dengan filter status
curl http://localhost:8000/api.php?status=upcoming

# Dengan filter tanggal
curl "http://localhost:8000/api.php?date_from=2025-11-01&date_to=2025-12-31"
```

Response:

```json
```

```
{
  "status": 200,
  "message": "Events retrieved successfully",
  "data": [
    {
      "id": "1",
      "title": "Workshop Laravel",
      "date": "2025-11-15",
      "time": "09:00:00",
      "location": "Jakarta Convention Center",
      "description": "Belajar Laravel dari dasar",
      "capacity": 100,
      "status": "upcoming",
      "created_at": "2025-10-24 10:00:00",
      "updated_at": "2025-10-24 10:00:00"
    }
  ],
  "timestamp": "2025-10-24 15:30:00"
}
```


2. GET Single Event

```bash
curl http://localhost:8000/api.php?id=1
```

3. GET Statistics

```bash
curl http://localhost:8000/api.php?stats=1
```

Response:

```json
{
  "status": 200,
  "message": "Statistics retrieved successfully",
  "data": {
    "total": 15,
    "upcoming": 8,
    "ongoing": 2,
    "completed": 4,
    "cancelled": 1
  }
}
```

4. POST Create Event

```bash
curl -X POST http://localhost:8000/api.php \
-H "Content-Type: application/json" \
-d '{
  "title": "Workshop React",
  "date": "2025-12-01",
  "time": "14:00:00",
  "location": "Tech Hub Jakarta",
  "description": "Belajar React untuk pemula",
  "capacity": 50,
  "status": "upcoming"
}'
```

Response:

```json
{
  "status": 201,
  "message": "Event created successfully",
  "data": {
    "id": "6",
    "title": "Workshop React",
    ...
  }
}
```

5. PUT Update Event

```bash
curl -X PUT http://localhost:8000/api.php?id=1 \
-H "Content-Type: application/json" \
-d '{
  "title": "Workshop Laravel Advanced",
  "date": "2025-11-16",
  "time": "10:00:00",
  "location": "Jakarta Convention Center",
  "description": "Laravel level advanced",
  "capacity": 80,
  "status": "upcoming"
}'
```

6. DELETE Event

```bash
curl -X DELETE http://localhost:8000/api.php?id=1
```

📋 Field Validation

Required Fields

- `title` (string, max 255)
- `date` (date, format: YYYY-MM-DD)
- `time` (time, format: HH:MM:SS)
- `location` (string, max 255)
- `status` (enum: upcoming, ongoing, completed, cancelled)

Optional Fields

- `description` (text)
- `capacity` (integer, >= 0)

🔒 Status Codes

Code	Meaning
200	OK - Request berhasil
201	Created - Resource berhasil dibuat

Code	Meaning
400	Bad Request - Data invalid
404	Not Found - Resource tidak ditemukan
405	Method Not Allowed
500	Internal Server Error

💡 Contoh Integrasi JavaScript

```javascript
```

```javascript
// Fungsi helper untuk API
const API_URL = 'http://localhost:8000/api.php';

// Get all events
async function getAllEvents() {
  const response = await fetch(API_URL);
  const data = await response.json();
  return data.data;
}

// Create event
async function createEvent(eventData) {
  const response = await fetch(API_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(eventData)
  });
  return await response.json();
}

// Update event
async function updateEvent(id, eventData) {
  const response = await fetch(`${API_URL}?id=${id}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(eventData)
  });
  return await response.json();
```

```
}

// Delete event
async function deleteEvent(id) {
  const response = await fetch(`${API_URL}?id=${id}`, {
    method: 'DELETE'
  });
  return await response.json();
}

// Get statistics
async function getStats() {
  const response = await fetch(`${API_URL}?stats=1`);
  const data = await response.json();
  return data.data;
}

// Contoh penggunaan
async function demo() {
  // Buat event baru
  const newEvent = await createEvent({
    title: 'Tech Meetup',
    date: '2025-12-15',
    time: '18:00:00',
    location: 'Coworking Space',
    description: 'Diskusi teknologi terkini',
    capacity: 30,
    status: 'upcoming'
  });

  console.log('Event created:', newEvent);
```

```
  // Ambil semua event
  const events = await getAllEvents();
  console.log('All events:', events);

  // Ambil statistik
  const stats = await getStats();
  console.log('Statistics:', stats);
}
```

🎨 Contoh Integrasi PHP (Frontend)

php

```php
<?php
// Simple PHP frontend example
$apiUrl = 'http://localhost:8000/api.php';

// Get events
$response = file_get_contents($apiUrl);
$result = json_decode($response, true);
$events = $result['data'];

foreach ($events as $event): ?>
  <div class="event-card">
    <h3><?= htmlspecialchars($event['title']) ?></h3>
    <p>📅 <?= $event['date'] ?> | ⏰ <?= $event['time'] ?></p>
    <p>📍 <?= htmlspecialchars($event['location']) ?></p>
    <span class="badge"><?= $event['status'] ?></span>
  </div>
<?php endforeach; ?>
```

🔧 Tips & Best Practices

1. **Security**
   - Gunakan prepared statements (sudah diimplementasi)
   - Validasi semua input
   - Gunakan HTTPS di production
   - Implementasi authentication/authorization

2. **Performance**
   - Tambah index pada kolom yang sering di-query
   - Gunakan caching untuk read-heavy operations
   - Implement pagination untuk data besar

3. **Monitoring**
   - Log semua error ke file
   - Monitor database performance
   - Setup automatic backups

4. **Testing**

```bash
# Test dengan curl
./test_api.sh
```

📞 Error Handling

Semua error dikembalikan dengan format:

```json
{
  "status": 400,
  "message": "Error description",
  "data": null,
  "timestamp": "2025-10-24 15:30:00"
}
```

🚀 Production Deployment

1. Enable error logging
2. Disable display_errors di php.ini
3. Setup SSL certificate
4. Configure database connection pooling
5. Implement rate limiting
6. Setup monitoring dan alerts

📝 Changelog

v1.0.0 - Initial release

- CRUD operations

- MySQL integration
- Statistics endpoint
- Input validation